CASSANDRA VERITAS

Signals Under the Surface

Combined Technical Report — All Parts

Task	Metric	Platform
Auto Insurance Claim Prediction	Normalized Gini Coefficient	Kaggle

Team: XYZ **Members:** Soham Labhshetwar & Naitik Agrawal

Abstract

This report presents a complete end-to-end pipeline for predicting the probability that an auto insurance policyholder will file a claim, using an anonymized dataset of driver behavior and vehicle characteristics provided by Veritas Risk Corp. The solution is structured across three phases: **Part A** performs thorough exploratory data analysis covering feature metadata, target distribution, and missing value patterns; **Part B** conducts structural and advanced discovery through representation learning (PCA, UMAP), network analysis (feature correlation graphs, community detection), interaction mining (mutual information, SHAP), and a proposed data generation pipeline; **Part C** delivers the final predictive model — an Optuna-optimized ensemble of LightGBM, XGBoost, and CatBoost trained under 5-fold stratified cross-validation, evaluated via the Normalized Gini Coefficient.

1 Part A — Data Investigation

1.1 A1 — Feature Understanding and Metadata

The dataset contains 58 predictive features (excluding `id` and `target`) organized into four semantically meaningful groups based on their naming prefix. This structure reflects the domain — each group captures a different dimension of driver and policy risk:

- `ps_ind` — Individual/personal driver attributes (age, driving record proxies)
- `ps_reg` — Regional characteristics (geographic risk factors)
- `ps_car` — Vehicle-related attributes (car type, age, value proxies)
- `ps_calc` — Calculated/engineered features (pre-computed risk indicators)

Feature types are further encoded via name suffixes, enabling automated classification:

Type	Suffix	Count	Description
Binary	_bin	17	Boolean indicators (0 or 1)
Categorical	_cat	14	Nominal multi-level variables
Continuous	(none)	27	Real-valued or ordinal features
Total	—	**58**	—

Group-level breakdown shows `ps_calc` is the largest group (20+ features, mostly continuous), while `ps_reg` is the smallest with only 3 features. The `ps_car` group contains the most categorical features, making it critical for careful encoding. All features are anonymized, so interpretation relies on statistical behavior rather than domain knowledge.

1.2 A2 — Target Distribution and Feature Distributions

Critical Finding: Severe Class Imbalance

The target variable is highly imbalanced. Approximately **96.4%** of policyholders filed no claim (class 0) versus only **3.6%** who did (class 1), giving a class ratio of approximately **26.7:1**. This severe imbalance is the most important structural property of the dataset and fundamentally shapes every modeling decision downstream — from the choice of evaluation metric (Gini over accuracy) to training strategies (class weighting, stratified splits).

1.2.1 Continuous and Ordinal Feature Distributions

Overlaid KDE/histogram plots of continuous features split by target class revealed that features in the `ps_car` and `ps_ind` groups exhibit the strongest distributional separation between claimers (target=1) and non-claimers (target=0). Several continuous features showed right-skewed distributions with heavier tails for the claim class, suggesting that extreme feature values are disproportionately associated with filing a claim.

1.2.2 Categorical Feature Distributions

For each categorical feature, the claim rate per category was plotted against the overall claim rate baseline. Key observations:

- Some categories exhibit claim rates up to 3× the overall mean, confirming strong non-uniform risk stratification within categorical variables.
- The feature `ps_ind_05_cat` showed particularly stark variation across categories and was consistently identified as highly discriminative.
- Several categories in `ps_car_*_cat` features also showed elevated claim rates, reinforcing the importance of vehicle-related attributes.

1.2.3 Binary Feature Analysis

Binary features were analyzed by comparing claim rates for value=0 vs. value=1. Multiple `ps_ind_bin` features showed systematically higher claim rates when their value equals 1, suggesting they flag specific driver attributes associated with elevated risk. These features, while simple, provide clean and reliable signal.

1.3 A3 — Missing Values and Feature Relationships

Missing values throughout the dataset are encoded as -1 rather than `NaN`. This requires explicit handling — values must be replaced with `NaN` before any statistical analysis or model training. The features with missing values and their missingness rates are:

Feature	Missing Count	Missing %
`ps_car_03_cat`	102,642	69.09%
`ps_car_05_cat`	66,566	44.82%
`ps_reg_03`	26,896	18.12%
`ps_car_14`	10,680	7.19%
`ps_car_07_cat`	2,804	1.89%
`ps_ind_02_cat`	216	0.15%
`ps_ind_04_cat`	83	0.06%
`ps_ind_05_cat`	107	0.07%

Missingness as predictive signal: Claim rate analysis comparing missing vs. non-missing observations revealed that `ps_car_03_cat` and `ps_car_05_cat` show meaningfully different claim rates when the value is missing versus observed. This indicates the missingness itself is *not* random (Missing Not At Random, MNAR) and carries predictive information. Binary missingness indicator flags were therefore created as additional features for these columns.

Missingness co-occurrence: A correlation heatmap of missingness indicators revealed that `ps_car_03_cat` and `ps_car_05_cat` tend to be missing together, suggesting a shared underlying cause — for example, a specific vehicle category for which both attributes are unavailable.

Imputation strategy: Median imputation was applied to continuous/ordinal features and mode imputation to binary and categorical features. All fill values were computed on the training set and applied identically to the test set to prevent data leakage.

Feature correlation structure: Spearman rank correlation analysis with the target variable identified `ps_car`

features as most correlated overall. In contrast, many `ps_calc` features showed near-zero Spearman correlation, suggesting limited individual linear signal. However, these features were retained since tree-based models can exploit non-linear and interaction-based contributions.

2 Part B — Structural Discovery and Advanced Exploration

2.1 B1 — Representation Learning

The goal of representation learning is to project driver profiles into a lower-dimensional space that captures meaningful structure not immediately visible in the raw feature space.

2.1.1 Principal Component Analysis (PCA)

PCA was applied to the standardized feature matrix (zero mean, unit variance). The scree plot showed a gradual, nearly linear decay in explained variance with no sharp elbow, indicating the data does not compress easily into a small number of components:

- Approximately **35 principal components** are needed to explain 90% of total variance.
- Approximately **45 principal components** are needed to explain 95% of total variance.

This suggests the dataset has moderately high intrinsic dimensionality and naive reduction (keeping only 2–5 PCs) would discard significant information. The 2D PCA projection, colored by target class, shows substantial overlap between claim and no-claim populations — confirming the task is not linearly separable. A slight density shift along PC1 suggests some weak linear structure is present.

2.1.2 UMAP Embedding

UMAP (Uniform Manifold Approximation and Projection) was applied with `n_neighbors=15` and `min_dist=0.1` on a subsample of 15,000 drivers for computational tractability. Unlike PCA, UMAP preserves both local and global non-linear structure.

UMAP Finding

The 2D UMAP embedding revealed several distinct density clusters within the driver population. Claim=1 drivers (the positive class) are **not uniformly distributed** across the manifold — they concentrate within specific regions. This confirms that exploitable non-linear structure exists in the data, and that driver subpopulations with genuinely different risk profiles are present.

2.2 B2 — Relationship and Network Discovery

2.2.1 Feature Correlation Network

A graph $G = (V, E)$ was constructed where nodes V represent the 58 features and edges E connect pairs with absolute Spearman correlation exceeding a threshold of 0.15. This threshold was chosen to retain meaningful relationships while avoiding a fully dense graph.

- Hub features (high degree centrality) are concentrated in `ps_car` and `ps_ind`, consistent with their higher target correlation found in Part A.
- Louvain community detection identified **4–6 feature communities**, broadly aligning with the four semantic groups — validating the dataset's internal design consistency.
- `ps_calc` features formed a loosely connected subgraph with few inter-group edges, consistent with their low target correlation and potentially independent origin.

2.2.2 Driver Risk Clustering

K-Means clustering was applied on the UMAP embedding coordinates, with optimal K selected by maximizing the silhouette score over $K \in \{2, \ldots, 8\}$. Clusters were characterized by their claim rates relative to the overall mean:

Cluster	Risk Profile	Claim Rate (relative)	Population
C0	Low Risk	$< 0.5\times$ overall mean	Large
C1	Moderate Risk	$\approx$ overall mean	Large
C2	High Risk	1.5–2.5× mean	Medium
C3	Anomalous Risk	$> 2.5\times$ mean	Small

The *anomalous risk* cluster (C3) represents a small but highly concentrated population of high-risk drivers whose characteristics differ significantly from the majority. Identifying this segment has practical implications beyond modeling: it informs premium pricing, underwriting decisions, and fraud detection strategies.

2.3 B3 — Interaction and Dependency Discovery

Since all features are anonymized, individual feature names carry no semantic meaning. Meaningful signal may only emerge when features are considered jointly. This section investigates such interactions systematically.

2.3.1 Mutual Information Analysis

Mutual information (MI) between each feature and the binary target was estimated using a k-nearest-neighbor estimator with $k = 5$. Unlike Pearson or Spearman correlation, MI captures both linear and non-linear dependencies. Key findings:

- `ps_car` features dominate the top-MI ranking, consistent with their Spearman correlation results from Part A.
- Several `ps_calc` features have near-zero MI with the target, suggesting they contribute little individually. They are nonetheless retained in case they participate in relevant interaction terms.
- MI rankings broadly agree with Spearman correlation results but expose several features whose non-linear relationship with the target exceeds what correlation alone captures.

2.3.2 SHAP Feature Importance and Interaction Analysis

A LightGBM model was trained on the full training set and SHAP (SHapley Additive exPlanations) values were computed on a held-out validation subset. SHAP provides theoretically grounded, per-prediction feature attribution based on cooperative game theory.

Key findings:

- Top features by mean |SHAP|: `ps_car_13`, `ps_ind_05_cat`, `ps_car_14`, `ps_reg_03`.
- SHAP distributions reveal non-linear, asymmetric effects: high values of certain features dramatically increase predicted claim probability, while low values have negligible effect — consistent with threshold-like risk mechanisms.
- Several features exhibit bidirectional SHAP effects, where contribution direction reverses depending on other feature values — a direct signature of interaction effects.

2.3.3 Pairwise Feature Interaction Testing

Pairwise product terms $f_i \times f_j$ were constructed for the top-6 SHAP features and added individually to the feature set. LightGBM was retrained with each interaction term, and AUC lift over the baseline was measured. Positive AUC lifts were observed for select pairs, confirming the existence of non-additive feature interactions. These terms were incorporated into the final model's feature engineering pipeline.

2.4 B4 — Statistical Structure and Data Generation

Note: Code is not required for this section per the problem statement. The pipeline is described conceptually.

2.4.1 Objective

Model the joint distribution $p(x_1, x_2, \ldots, x_p, y)$ over the mixed-type feature space (binary, categorical, continuous) and reproduce synthetic driver profiles statistically faithful to the observed dataset.

2.4.2 Proposed Pipeline

Step 1 — Preprocessing: Replace all -1 encoded missing values with `NaN`. Apply mode/median imputation. Label-encode categorical features to integers.

Step 2 — CTGAN (Primary Approach): Fit a Conditional Tabular GAN using the `ctgan` Python library, specifically designed for tabular data with mixed types. It uses mode-specific normalization for continuous features and a conditional vector to ensure the minority class (claim=1) is sampled with correct frequency.

Step 3 — Gaussian Copula (Alternative): Fit per-feature marginal distributions and model inter-feature dependencies via a Gaussian copula. More interpretable and computationally cheaper, but assumes Gaussian dependence structure.

Step 4 — Validation Protocol:

- *Marginal fidelity:* Per-feature KS-test between real and synthetic distributions.
- *Classifier test:* Train a discriminator classifier (real=1 vs. synthetic=0). AUC near 0.5 indicates high-quality

synthesis.

- *Downstream utility:* Train a claim predictor on synthetic data only, evaluate on real held-out test set.

Use cases: Class balancing augmentation, stress-testing models on out-of-distribution profiles, and privacy-preserving data sharing for research.

3 Part C — Predictive Modeling

3.1 C1 — Preprocessing and Feature Engineering

All preprocessing steps were designed with strict train/test separation to prevent data leakage. Fill values, encoders, and aggregate statistics were computed exclusively on the training set and applied identically to test.

3.1.1 Data Cleaning

- **NaN target removal:** One row in the training set contained a `NaN` target value and was dropped before any processing.
- **Missing value encoding:** All -1 values replaced with `NaN`, followed by mode imputation for binary/categorical features and median imputation for continuous features.
- **Missingness indicator flags:** Binary columns added for each feature with $> 1\%$ missingness, encoding whether the original value was missing — capturing the MNAR signal identified in Part A.

3.1.2 Feature Engineering

- **Group-level aggregates:** For each of the four feature groups, five summary statistics (sum, mean, standard deviation, maximum, minimum) were computed across continuous features in that group. These aggregate features capture group-level driver profiles.
- **Row-level missing count:** Total number of originally missing features per driver row, added as a continuous feature.
- **Binary feature sum:** Sum of all binary features per row, serving as a composite risk score proxy.

Final feature count after engineering: approximately **85 features**.

3.2 C2 — Model Architecture

Three gradient-boosted tree models were selected for their complementary strengths on tabular data with class imbalance. All three were trained independently under the same 5-fold stratified cross-validation framework.

3.2.1 Model 1: LightGBM

LightGBM was the primary model, chosen for its speed, memory efficiency, and strong performance on imbalanced tabular datasets. A custom evaluation function implementing the Normalized Gini Coefficient was passed as `feval`, enabling early stopping on the actual competition metric rather than a proxy like AUC.

Hyperparameter	Value
`num_leaves`	63
`learning_rate`	0.02
`feature_fraction`	0.7
`bagging_fraction`	0.8
`bagging_freq`	5
`reg_alpha / reg_lambda`	0.1 / 1.0
`scale_pos_weight`	$\approx$26.7 (imbalance ratio)
`num_boost_round`	Up to 3000 (early stopping at 100)

3.2.2 Model 2: XGBoost

XGBoost provides strong regularization via L1/L2 penalties and column subsampling, offering a complementary bias-variance tradeoff to LightGBM.

Hyperparameter	Value
`max_depth`	6
`eta`	0.02
`colsample_bytree`	0.7
`subsample`	0.8
`alpha / lambda`	0.1 / 1.0
`scale_pos_weight`	≈26.7
`tree_method`	`hist` (GPU-compatible)
`num_boost_round`	Up to 3000 (early stopping at 100)

3.2.3 Model 3: CatBoost

CatBoost natively handles categorical features without label encoding, using internal ordered target statistics encoding that reduces target leakage during training. This makes it particularly well-suited to this dataset's 14 categorical features.

Hyperparameter	Value
`depth`	6
`learning_rate`	0.02
`l2_leaf_reg`	3
`bagging_temperature`	0.8
`scale_pos_weight`	≈26.7
`iterations`	Up to 3000 (early stopping at 100)

3.3 C3 — Validation Strategy and Evaluation Metric

5-Fold Stratified Cross-Validation: Each fold maintains the original 96.4/3.6 class ratio. Out-of-fold (OOF) predictions are collected across all folds to form a complete OOF prediction array, on which the final Normalized Gini is computed. This gives a more stable and unbiased estimate of generalization performance than a single validation split.

3.3.1 Normalized Gini Coefficient

The Normalized Gini Coefficient is the primary evaluation metric. It measures how well the model ranks observations from highest to lowest predicted risk, comparing the resulting ordering against a theoretical perfect ranking:

$$G_{\text{norm}} = \frac{G_{\text{raw}}}{G_{\text{max}}}, \qquad G_{\text{max}} = \frac{1 - f_{\text{pos}}}{2}$$

where f_{pos} is the fraction of positive class observations and G_{raw} is computed by sorting by descending predicted probability and comparing the cumulative positive rate against a uniform baseline. A score of 0 corresponds to random guessing; 1.0 is the theoretical maximum. The relationship to AUC is: $G_{\text{norm}} = 2 \cdot \text{AUC}_{\text{norm}} - 1$.

3.4 C4 — Ensemble and Final Results

The three models' OOF predictions were blended via a weighted average. Ensemble weights were optimized using Optuna (TPE sampler) over 200 trials, maximizing OOF Gini:

$$\hat{y}_{\text{ensemble}} = w_1 \cdot \hat{y}_{\text{LGB}} + w_2 \cdot \hat{y}_{\text{XGB}} + (1 - w_1 - w_2) \cdot \hat{y}_{\text{CAT}}$$

Model	OOF Gini	Std (folds)	Weight
LightGBM	0.23539 ± 0.02032	5	$w_1 = 0.102$
XGBoost	0.23971 ± 0.01967	5	$w_2 = 0.124$
CatBoost	0.25730 ± 0.02038	5	$1 - w_1 - w_2 = 0.774$
Ensemble	**0.25859**	—	—

OOF Gini values from the completed Colab run. Std computed across 5 folds.

Why an Ensemble of Three Models?

Each model captures the data differently. LightGBM is fast and excels with `scale_pos_weight` handling; XGBoost offers strong L1/L2 regularization; CatBoost natively handles categorical features without label encoding artifacts. Their prediction errors are partially uncorrelated, so a weighted blend reduces variance and consistently outperforms any single model — a well-established result in competitive machine learning.

3.5 C5 — Key Design Decisions and Justifications

1. **Class imbalance via `scale_pos_weight`:** Rather than resampling (SMOTE, undersampling), native class weighting was used in all three models. This preserves the original data distribution and avoids artefacts from synthetic minority oversampling.
2. **Missingness as a feature:** Binary indicator columns encoding whether each value was originally missing were added to the feature set, motivated by the MNAR analysis in Part A where certain missing patterns correlated with higher claim rates.
3. **Retaining `ps_calc` features:** Despite near-zero individual MI and Spearman correlation, `ps_calc` features were retained. Tree-based models can detect interaction-based contributions invisible in marginal analysis.
4. **Stratified K-Fold:** With a 26.7:1 class ratio, random splits risk producing folds with very few positive examples. Stratified splitting guarantees each fold reflects the overall class distribution.
5. **Early stopping on Gini:** LightGBM was configured with a custom Gini evaluation function, so early stopping is driven by the actual competition metric rather than a proxy.
6. **Optuna ensemble optimization:** Rather than fixed equal weights, Optuna explored the weight space efficiently using TPE sampling, finding the optimal blend maximizing OOF Gini.

Raw CSV Data

↓

[Part A] Drop NaN targets · Replace $-1 \rightarrow$ NaN · Impute (median/mode) · Analyze missingness

↓

[Part B] PCA + UMAP embeddings · Network communities · SHAP interactions · Cluster risk profiles

↓

[Part C] Missingness flags · Group aggregates · Row-level features (≈85 total)

↓

[Part C] LightGBM + XGBoost + CatBoost under 5-Fold Stratified CV

↓

[Part C] Optuna-optimized weighted ensemble $\longrightarrow$ **Submission CSV**